# DSA Practice Problems - Multiples of Three

## Problem 1: Find All Multiples of 3

You are given an array A of N integers. Return all numbers from the array that are divisible by 3. If no such number exists, print -1.

### *Input Format:*

First line: Integer N

Second line: N space-separated integers

### *Output Format:*

Print all multiples of 3 in the same order as input.

If none exist, print -1.

### *Constraints:*

1 ≤ N ≤ 10^5

-10^9 ≤ A[i] ≤ 10^9

### *Example:*

```
Input:
7
2 3 6 7 9 10 12

Output:
3 6 9 12
```

```
##Steps to complete this problem
#Read input
#Traverse array once
#Check num % 3 == 0
#Store valid numbers
#If empty → print -1

def find_multiples_of_three():
    try:
        N = int(input().strip())
        arr = list(map(int, input().split()))

        if len(arr) != N:
            print("Invalid input")
            return

        multiples = []

        for num in arr:
            if num % 3 == 0:
                multiples.append(num)

        if multiples:
            print(*multiples)
        else:
            print(-1)

    except Exception:
        print("Invalid input")


# Call function
find_multiples_of_three()
```

```
7
2 3 6 7 9 10 12
3 6 9 12
```

## Problem 2: Longest Consecutive Multiples of 3

You are given an array A of N integers. Find the length of the longest consecutive sequence (continuous in array) where all elements are divisible by 3.

### *Input Format:*

First line: Integer N

Second line: N space-separated integers

### *Output Format:*

Print the length of the longest consecutive block of multiples of 3.

### *Example:*

```
Input:
10
3 6 9 2 12 15 18 5 21 24

Output:
3
```

```
#two variables : current_count& max_count
#current_count: counts current consecutive multiples
#Max_count stores maximum found so far

def longest_consecutive_multiples_of_three():
    N = int(input().strip())
    arr = list(map(int, input().split()))

    current_count = 0
    max_count = 0

    for num in arr:
        if num % 3 == 0:
            current_count += 1
            max_count = max(max_count, current_count)
        else:
            current_count = 0

    print(max_count)


# Function call
longest_consecutive_multiples_of_three()
```

```
10
3 6 9 2 12 15 18 5 21 24
3
```

# DSA Practice Problem - Cutting Trees Optimally

## Problem Statement:

You are given N trees arranged in a straight line. Each tree has a height H[i]. You are allowed to cut a tree only if its height is strictly greater than both of its immediate neighbors. When you cut a tree: - You gain wood equal to its height. - The tree is removed from the line. - The neighbors become adjacent. Your task is to determine the maximum total wood you can collect.

### *Input Format:*

First line: Integer N

Second line: N space-separated integers H[i]

### *Output Format:*

Print the maximum wood collected.

### *Constraints:*

1 ≤ N ≤ 10^5

1 ≤ H[i] ≤ 10^9

### *Example:*

```
Input:
5
2 5 3 4 1

Output:
9
```

### *Explanation:*

Cut 5 first (gain 5). New array: 2 3 4 1

Then cut 4 (gain 4). New array: 2 3 1

Total wood collected = 5 + 4 = 9

```
def max_wood_collected():
    N = int(input().strip())
    arr = list(map(int, input().split()))

    total_wood = 0

    while True:
        peaks = []

        # Find all current peaks
        for i in range(1, len(arr) - 1):
            if arr[i] > arr[i - 1] and arr[i] > arr[i + 1]:
                peaks.append((arr[i], i))

        # If no peaks, stop
        if not peaks:
            break

        # Select the largest peak
        peaks.sort(reverse=True)
        height, index = peaks[0]

        total_wood += height

        # Remove that tree
        arr.pop(index)

    print(total_wood)


# Call function
max_wood_collected()
```

```
5
2 5 3 4 1
12
```

In this problem, after cutting a peak tree, the remaining trees become adjacent and newly formed peaks are re-evaluated dynamically. By repeatedly identifying and removing the largest valid peak until no peaks remain, we maximize the total wood collected. For the input (2 5 3 4 1), the trees 5, 4, and 3 can be cut sequentially, resulting in a total wood collection of 12. This approach ensures optimal results with a time complexity of O(N log N).

```
def max_wood_original_peaks():
    N = int(input().strip())
    arr = list(map(int, input().split()))

    # If less than 3 trees, no peak possible
    if N < 3:
        print(0)
        return

    total_wood = 0

    # Check only original peaks
    for i in range(1, N - 1):
        if arr[i] > arr[i - 1] and arr[i] > arr[i + 1]:
            total_wood += arr[i]

    print(total_wood)


# Call function
max_wood_original_peaks()
```

```
5
2 5 3 4 1
9
```

In this problem, a tree is considered eligible for cutting only if it is a local peak in the original array, meaning its height is strictly greater than its immediate neighbors. By identifying and summing all such original peaks, we determine the maximum wood that can be collected. For the given example (2 5 3 4 1), the valid peaks are 5 and 4, resulting in a total wood collection of 9. The solution runs in O(N) time complexity.